**TASK: Implement a Complete, Secure, Free Manual KYC Verification System for My Gold Loan Jewellery Website**

Act as a senior full-stack developer and security engineer. Implement a complete, production-oriented KYC (Know Your Customer) verification module in my existing jewellery website.

**1. Existing Technology Stack**

- Frontend: React
- Backend: Node.js and Express
- Database: MySQL
- Database library: Use the existing MySQL connection pool and project conventions.
- Authentication: Reuse the existing login, JWT/session and role-based authorization system.
- Deployment: Preserve the existing hosting and deployment configuration.

IMPORTANT: First inspect the entire project structure, existing routes, controllers, database schema, authentication middleware, file-upload system, frontend routing, admin dashboard and coding conventions. Do not blindly create duplicate files, tables, authentication systems or database connections.

Before editing, summarize the existing architecture and implementation plan. Then implement the complete feature directly in the existing project.

**2. Main Objective**

Build a free manual KYC workflow now that can support automatic KYC API integration in the future.

The manual workflow must not require a paid verification API. Hosting, storage and other infrastructure may still incur costs.

The system must support customer identity-document submission, private document storage, admin review, approval, rejection, resubmission and KYC status tracking.

Design the backend so an authorized KYC provider can be integrated later without rewriting the entire application.

Do not claim that uploading a document or manually reviewing it automatically establishes its authenticity.

**3. Customer KYC Workflow**

Implement a responsive React KYC page integrated with the existing customer account.

The page must include:

- Customer information populated from the authenticated account where possible.
- Required document type selection based on the applicable lender's approved KYC policy.
- Secure document upload.
- Clear document format and file-size requirements.
- A privacy notice and explicit consent.
- Submission confirmation.
- Current verification status.
- Submission date and review date.
- Admin rejection reason and resubmission instructions.
- A resubmission feature for eligible applications.
- Loading states, validation messages, API error handling and success notifications.

Support only document types appropriate for the applicable KYC policy. Do not assume every identity document is acceptable for every lender or verification process.

Validate required fields on both frontend and backend.

Prevent accidental duplicate submissions and multiple concurrent applications where inappropriate.

Do not store passwords, complete identity-document contents, or unnecessary sensitive personal information in browser localStorage.

**4. Secure Document Upload**

Implement secure document uploads using the existing upload conventions or a dedicated KYC upload middleware.

Requirements:

- Accept only permitted PDF, JPEG and PNG files.
- Enforce a configurable file-size limit, initially 5 MB per file.
- Validate actual file signatures/content, not only the filename or browser-provided MIME type.
- Generate random server-side filenames.
- Never trust a filename or file path supplied by the client.

- Prevent path traversal and unauthorized file access.
- Store documents outside the publicly accessible web root or in private object storage.
- Never expose the KYC storage directory using express.static().
- Do not store public document URLs.
- Provide a secure authenticated endpoint for authorized document viewing.
- Restrict document access to the document owner and authorized staff.
- Apply upload rate limits and appropriate request validation.
- Clean up temporary and abandoned uploads.
- Use encryption and secure storage controls appropriate for sensitive identity documents.
- Configure safe response headers when serving files.

If malware scanning or other production security controls are unavailable, document the limitation and provide a safe deployment checklist.

Never collect Aadhaar details or documents unnecessarily. Implement Aadhaar-related functionality only when legally permitted and supported by the relevant authorized process.

**5. MySQL Database**

Inspect the current database before designing or applying migrations.

Create or adapt the following logical entities as appropriate:

A. KYC applications:

- Application ID
- Existing authenticated user ID
- Applicant type or customer category if required
- Verification status
- Consent timestamp
- Consent/privacy-notice version
- Submission timestamp
- Reviewer ID
- Review timestamp

- Rejection or resubmission reason
- Provider name, initially "manual"
- Optional external provider reference
- Created and updated timestamps

B. KYC documents:

- Document ID
- Application ID
- Document type
- Private storage key or relative path
- Upload timestamp
- Necessary document metadata only

C. KYC audit history:

- Application ID
- Actor ID
- Previous status
- New status
- Action and review reason
- Timestamp

Add appropriate indexes, constraints, foreign keys and uniqueness rules after checking the existing schema.

Use database migrations or safe SQL migration scripts consistent with the project. Do not drop or overwrite existing production tables.

Use parameterized SQL queries throughout.

Use transactions for application submission and status transitions. Ensure failed transactions do not leave orphaned records or files.

Avoid storing complete ID numbers, document contents or unnecessary sensitive data in application logs.

**6. Verification Statuses**

Implement these normalized statuses:

- not_submitted
- pending
- in_review
- verified
- rejected
- resubmission_required
- failed, only where appropriate for future automated checks

Define valid state transitions and enforce them on the backend.

A customer must never be able to change their own status to verified.

An ordinary admin must not be able to bypass required permissions.

A verified status must only be assigned after the applicable verification procedure has actually been completed.

Keep KYC verification separate from gold-loan approval, gold valuation, credit decisions and loan disbursement.

**7. Backend API Endpoints**

Adapt the endpoint names to existing project conventions and implement all required controllers and validation.

Customer endpoints:

- POST /api/kyc — submit a KYC application and permitted documents.
- GET /api/kyc/status — get the authenticated user's KYC status.
- GET /api/kyc/documents/ — securely access an authorized document, if permitted.
- POST /api/kyc/resubmit — resubmit a rejected or eligible application.

Admin endpoints:

- GET /api/kyc/admin/applications — list applications with pagination and filters.
- GET /api/kyc/admin/applications/ — view authorized application details.
- PATCH /api/kyc/admin/applications//review — approve, reject or request resubmission.
- GET /api/kyc/admin/stats — return accurate status counts.

Requirements:

- Reuse existing authentication middleware.
- Add server-side role-based admin authorization to every admin endpoint.
- Obtain customer identity from the authenticated server session/token, never from a client-supplied user ID.
- Validate every parameter and request body.
- Implement pagination and safe filtering.
- Return consistent JSON responses and appropriate HTTP status codes.
- Avoid exposing stack traces, private storage paths or sensitive document information.
- Implement transaction-safe status changes and audit history.
- Prevent duplicate requests and unsafe status transitions.
- Use CSRF protection where applicable to the authentication model.

Do not leave controllers, imports, route handlers or required middleware undefined.

**8. Admin Dashboard**

Integrate KYC management into the existing admin dashboard.

Provide:

- Pending applications count.
- In-review applications count.
- Verified applications count.
- Rejected applications count.
- Search by appropriate customer identifiers.
- Filters by status and date.
- Pagination.
- Applicant and application details.
- Secure document viewing.
- Review notes.
- Approve, reject and request-resubmission actions.
- Confirmation before final decisions.
- Rejection reason validation.

- Reviewer information and audit history.
- Loading, empty, error and success states.

Use the project's existing UI framework, design system and responsive conventions. Do not introduce a new UI framework without a compelling reason.

The dashboard must retrieve data through authorized backend APIs, not access MySQL directly from React.

**9. Future Automatic KYC API Integration**

Implement a provider-independent service layer.

Suggested structure, adapted to the actual project:

- services/kyc/kycProvider.js
- services/kyc/manualProvider.js
- services/kyc/providerFactory.js

The manual provider must be the default.

Use an environment variable such as:

KYC_PROVIDER=manual

Provide a clearly documented interface for future providers that supports:

- Starting a verification session.
- Returning a provider reference.
- Normalizing provider responses.
- Processing signed webhooks or callbacks.
- Mapping results to internal statuses.
- Handling timeouts and retryable errors.
- Idempotent callback processing.
- Secure server-side credential configuration.

Do not invent provider API endpoints or claim automatic verification works before a real provider has been configured and tested.

Never expose provider credentials in React, browser bundles, source control or API responses.

For future Aadhaar or PAN verification, document the relevant authorization, consent, eligibility and provider-contract requirements.

**10. Privacy, Security and Compliance**

This system processes sensitive identity information.

Implement or document:

- Appropriate privacy notice and consent capture.
- Data minimization.
- Role-based access controls.
- Secure file storage and access logs.
- Encryption in transit and appropriate encryption at rest.
- Retention and secure deletion policies.
- Error handling that does not expose sensitive data.
- Audit records for every review decision.
- Rate limiting and abuse prevention.
- Secure configuration and secret management.
- Backup and recovery considerations.
- Applicable data-protection requirements and effective dates.

The website is intended for gold-loan applications. Do not assume a jewellery business is legally authorized to lend money or that generic manual KYC satisfies the applicable lender's obligations.

Document the need to confirm applicable Tamil Nadu licensing requirements and, where relevant, the regulated lender's KYC and gold-collateral rules before production launch.

Do not represent manual document upload as government-verified identity authentication.

**11. Frontend Integration**

Add the new pages and components using the existing project structure, for example:

- KycVerification.jsx
- KycStatus.jsx
- KycAdminDashboard.jsx
- KycApplicationDetails.jsx

These are suggested names only. Reuse existing components where suitable.

Integrate the pages with the current router, customer account area and admin navigation.

Use the existing API client and authentication mechanism.

Do not hardcode production API URLs, credentials, test results or customer information.

Make the UI accessible and responsive on mobile, tablet and desktop.

**12. Testing**

Add tests using the project's existing test framework.

Cover:

- Successful KYC submission.
- Missing consent.
- Invalid file type.
- Oversized file.
- Invalid file signature.
- Duplicate submission.
- Unauthenticated submission.
- Customer accessing another customer's application.
- Customer attempting to approve their own KYC.
- Non-admin attempting an admin action.
- Invalid status transitions.
- Successful admin review.
- Rejection and resubmission.
- Transaction rollback.
- Unauthorized document access.
- Path traversal attempts.
- Provider errors and duplicate webhook events in the future adapter.

Run existing tests and available lint/build commands.

Fix regressions caused by the implementation. Do not claim tests passed unless you actually ran them.

**13. Implementation and Delivery Instructions**

Work directly in my existing project.

Follow this sequence:

1. Inspect the repository and explain its architecture.
2. Identify existing authentication, user tables, admin roles, database pool and upload handling.
3. List the files to create or modify.
4. Implement database migrations.
5. Implement secure upload handling and KYC service interfaces.
6. Implement backend controllers, routes and authorization.
7. Implement customer-facing React pages.
8. Integrate the admin dashboard.
9. Implement validation, audit logging and error handling.
10. Add and run tests.
11. Run the frontend build and relevant backend checks.
12. Fix errors introduced by your changes.
13. Provide exact migration, configuration and deployment instructions.

Preserve existing functionality, payment processing, authentication, customer accounts, order management and admin features.

Do not replace the entire application or perform destructive database operations.

If an essential project detail is missing, inspect the code and ask only when necessary. Never fabricate credentials, existing table names or successful verification results.

At completion, provide:

- A summary of files created and modified.
- Database migration instructions.
- Required environment variables.
- Commands to run locally.
- API endpoint documentation.
- A manual testing checklist.

- Deployment instructions.
- Remaining security or compliance limitations.
- A clear explanation of how to replace the manual provider with an authorized automatic KYC provider later.

Begin by inspecting my existing repository, then implement the complete feature rather than returning only example code.